

Find the k -th Bit in the n -th Binary String

The sequence of binary strings (S_n) is defined as:

$$S_n = \begin{cases} "0", & \text{if } n = 1 \\ S_{n-1} + "1" + \text{reverse}(\sim(S_{n-1})), & \text{if } n > 1 \end{cases}$$

where $+$ is string concatenation, $\text{reverse}(s)$ is s with its characters in reverse order, and $\sim$ is the bitwise NOT operation (flips every 0 in the string to 1 and every 1 to 0).

Given two integers n and k , compute the k -th bit in S_n .

Example Test Cases:

Test Case 1

Input: $n = 3, k = 1$

Output: '0'

Explanation: S_3 is "0111001". The first bit is '0'.

Test Case 2

Input: $n = 4, k = 11$

Output: '1'

Explanation: S_4 is "011100110110001". The 11th bit is '1'.

Test Case 3

Input: $n = 1, k = 1$

Output: '0'

Explanation: S_1 is "0". The 1st bit is '0'.

Expected Time Complexity: $O(n)$

Input Format:

Two integers, n and k .

Output Format:

Return the k -th bit of the string S_n , defined above.

Constraints:

1. $1 \leq n \leq 40$
2. $1 \leq k \leq 2^n - 1$

(After reading this question, follow the steps described in the `Instructions.pdf` file to see the exact input and output format for the provided test cases.)

Instructions:

You have been provided with two template files, containing the `main` and `solve` functions respectively. The `main` function, which has been completed for you, parses the input for each test case. It then calls the `solve` function, which must compute the k -th bit of the n -th string, defined above.

The `solve` function must compute this in $O(n)$ time.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)

Library Usage:

For this lab, you are permitted to use any modules which are part of the standard library of either `C++` or `python`. For `C++`, do not add any extra `#include` directives; the provided `bits/stdc++.h` covers all allowed modules.